

Projeto de Software de Final de Disciplina

Aplicação para Controle de Vacinas de Animais PET baseada em Console User Interface (CUI)

Sumário

Apresentação.....	1
Organização dos Grupos.....	2
Datas Importantes.....	3
Console User Interface.....	3
Bibliotecas Java.....	4
Bibliotecas Python.....	4
Avaliação do Projeto.....	5
Critérios para o software.....	5
Critérios para a apresentação.....	5
Critérios para o vídeo.....	6

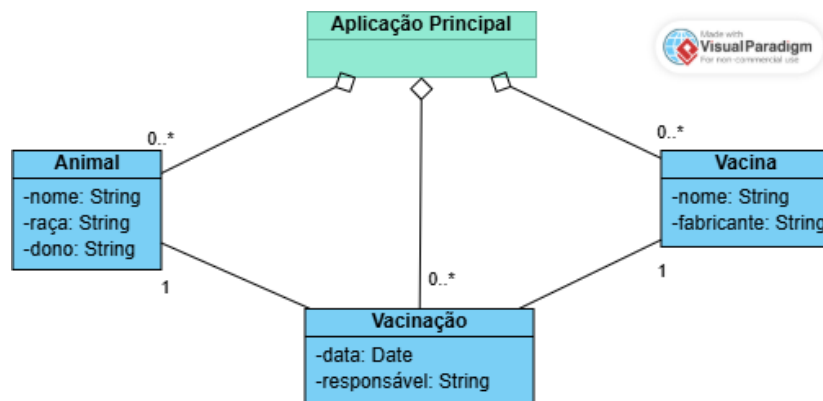
Apresentação

Considere uma clínica veterinária ou uma loja para *pets* (*Pet Shop*) que comercializa o serviço de vacinação. Nesse estabelecimento o controle das vacinas administradas em cada animal é rigoroso, ao ponto do tutor do animal conseguir emitir uma carteira de vacinação, ou seja, uma listagem de todas as vacinas administradas em um animal, incluindo detalhes como a data da vacinação.

Neste projeto de final de disciplina, cada equipe será desafiada a elaborar uma aplicação, ou seja, um software de registro das vacinas de um animal. Essa aplicação será capaz de responder precisamente, para um dado animal, “*quando e qual vacina foi administrada?*”.

As equipes precisarão identificar, a partir desse breve enunciado, quais classes e relações entre essas classes constituirá a aplicação. Para cada classe, quais atributos e métodos. Uma sugestão inicial de projeto dessas classes está representada na Figura 1. No entanto, outras classes, atributos, métodos ou relacionamentos poderão ser identificados.

Figura 1 - Proposta de Objetos da Aplicação



Assim, o software deve considerar como requisitos mínimos:

- Objetos esperados
 - Lista de vacinas
 - Lista de animais
 - Lista de vacinações
 - Da aplicação
 - Do animal
- Funcionalidades esperadas
 - Interface de cadastro de animal
 - Adiciona animal na lista
 - Interface de listagem de animais
 - Apresenta os animais da lista como uma tabela
 - Interface de cadastro de vacina
 - Adiciona vacina na lista
 - Interface de listagem de vacinas
 - Apresenta as vacinas da lista como uma tabela
 - Interface de registro de vacinação em um animal
 - Adiciona uma vacinação na lista de vacinações do animal
 - Interface de apresentação do histórico de vacinações de um animal
 - Apresenta as vacinações do animal como uma tabela

Organização dos Grupos

A turma será organizada em quatro equipes, inicialmente com formação estabelecida por sorteio. Porém, as equipes podem ser modificadas até o dia **20 de abril**.

Cada grupo será responsável pela criação da aplicação em uma das linguagens de programação adotadas no curso, ou seja, Java ou Python. A escolha da linguagem será definida por sorteio. Assim, teremos dois grupos com a linguagem Java e dois grupos com a linguagem Python.

Datas Importantes

Importante considerar o prazo de envio da tarefa cadastrada no SIGAA, dia **12/Julho**. No caso, a tarefa admite mais de uma submissão, tal que a última submissão substitui a submissão anterior. Ademais, é suficiente um membro do grupo realizar essa submissão, que será de um único arquivo compactado (ZIP ou 7z ou RAR), contendo os seguintes arquivos:

- Apresentação do projeto (slides) em PDF
- Arquivo texto com os seguintes conteúdos
 - Link de vídeo no **YouTube** de demonstração do software (até 3min de duração)
 - Link no **GitHub** ou **GitLab** do projeto público do software

A apresentação dos trabalhos será no dia **13/Julho**. Cada equipe terá até 15min de tempo de apresentação, organizadas como segue:

- Dois grupos em Python
- Dois grupos em Java

Assim, duas **datas importantes**:

- Domingo, **12/julho** - Prazo final de submissão da tarefa
- Segunda-feira, **13/julho** - Apresentações das equipes

Console User Interface

Console User Interface (CUI) é uma interface de texto que permite a interação entre o usuário e o software exclusivamente através de comandos e caracteres. Diferente das GUI (Graphic User Interface), ela **utiliza o terminal** para exibir dados e receber entradas, sendo ideal para focar na lógica de negócios e eficiência computacional.

A adoção de interfaces CUI em projetos acadêmicos prioriza o domínio da Lógica de Programação e dos pilares da Programação Orientada a Objetos (POO). Ao eliminar a complexidade de gerenciamento de eventos gráficos e design visual, o foco será na arquitetura de classes, herança e polimorfismo, garantindo sistemas robustos e funcionalmente bem estruturados.

As linguagens Java e Python possuem bibliotecas disponíveis para implementar, de forma mais rica, esse formato de interface baseada no terminal ou console. Assim, como opções de bibliotecas gratuitamente disponíveis, várias bibliotecas são citadas a seguir.

Bibliotecas Java

JLine: Especializada em lidar com a entrada do usuário no console. Oferece recursos avançados como histórico de comandos, autocompletar (tab completion) e edição de linha, sendo essencial para shells interativos.

- <https://jline.org/>
- <https://github.com/jline/jline3>

Lanterna: Uma biblioteca "pura Java" que permite criar interfaces tipo TUI (Terminal User Interface). Ela simula um terminal gráfico com janelas e botões, funcionando inclusive em ambientes sem suporte nativo a curses.

- **Site:** <https://github.com/mabe02/lanterna>

JCurses: Uma biblioteca que provê uma interface Java para a tradicional biblioteca *ncurses* do C. É ideal para criar janelas, menus e caixas de diálogo clássicas dentro do terminal Linux ou Unix.

- **Site:** <https://sourceforge.net/projects/javacurses/>

Bibliotecas Python

Rich: Focada em tornar o terminal visualmente atraente. Facilita a exibição de cores, tabelas formatadas, barras de progresso, sintaxe de código realçada e emojis com extrema simplicidade de código.

- <https://rich.readthedocs.io/en/stable/>
- <https://github.com/Textualize/rich>

Curses: O módulo padrão do Python para manipulação direta do terminal. É a ferramenta de baixo nível para controlar o posicionamento do cursor, cores de fundo e captura de teclas, sendo a base histórica das interfaces textuais.

- <https://docs.python.org/3/howto/curses.html>

Textual: Um framework TUI moderno e poderoso construído sobre a Rich. Ele utiliza conceitos de desenvolvimento web (como CSS) para criar interfaces complexas, reativas e com suporte total a mouse dentro do console.

- <https://github.com/Textualize/textual>

Avaliação do Projeto

A criação do projeto de software de final de disciplina é **parte** da avaliação do NAP II (Nota de Avaliação Parcial II). No caso, esse projeto representa **40% da nota final**. Para tanto, considerará alguns itens:

1. o software funcionando;
2. a apresentação pública pela equipe;
3. e o vídeo produzido.

Critérios para o software

Elemento mais importante da avaliação, pois é pré-requisito para avaliação da apresentação e do vídeo. Assim, o projeto deve contemplar todas as funcionalidades esperadas, usando linguagem de programação esperada, definida por sorteio.

A persistência dos dados dos objetos **não precisará** ser implementada, ou seja, os dados poderão ser salvos apenas em memória. No entanto, caso alguma equipe deseje implementar na aplicação o suporte a persistência dos dados, seja em arquivo, seja em banco de dados, esse esforço extra será considerado como diferencial na avaliação.

Da mesma forma, o esforço adicional para implementar funcionalidades extras na aplicação será considerado. Por exemplo, objeto tutor e lista de tutores, fabricante da vacina e lista de fabricantes da vacina, para ficar em dois exemplos.

Critérios para a apresentação

A apresentação terá duração de 15min e poderá ser realizada por apenas um dos membros da equipe. Outros membros poderão apresentar de forma conjunta, mas não é uma exigência. A apresentação será uma oportunidade da equipe fazer a defesa técnica, de forma sintética, do projeto desenvolvido. Assim, os seguintes critérios serão considerados:

- Domínio do Problema
 - A equipe demonstrou clareza ao explicar a dor do usuário e por que a solução proposta é relevante?
- Clareza da implementação/codificação
 - A explicação técnica foi coerente
- Qualidade da Demonstração
 - Nem sempre haverá tempo para esse momento, então pode ser substituído por vídeo ou imagens
 - O software fluiu bem? Apresentaram as funcionalidades críticas dentro do tempo?
- Gestão do Tempo e Postura
 - A equipe respeitou os limites de tempo?

- A equipe manteve uma comunicação profissional/segura?
- Qualidade das Respostas
 - Nem sempre haverá tempo para esse momento
 - A capacidade de responder a perguntas imprevistas sobre o código ou a lógica do negócio

Critérios para o vídeo

A produção e publicação do vídeo no YouTube é uma oportunidade, complementar a apresentação, para a equipe descrever o resultado final de seu projeto de software. Portanto, trata-se de uma evidência prática da experiência de uso do software funcionando, mas dentro de um “ambiente controlado”, narrando os principais cenários de uso do mesmo. Assim, os seguintes critérios serão considerados:

- Roteiro de Demonstração
 - O vídeo segue um fluxo lógico de uso do sistema (início ao fim do processo principal)?
- Destaque Funcional
 - O vídeo foca nas funcionalidades que agregam valor, sem se perder em menus irrelevantes?
- Qualidade Técnica (Áudio/Vídeo)
 - A gravação está nítida e o áudio da narração está compreensível?
- Sincronia com o Sistema
 - O software rodando no vídeo condiz com a descrição técnica apresentada na apresentação?